Class 5 · User stories, estimation and velocity

writing the backlog like a professional, and the first numbers of the course

Monday, October 19, 2026 · Sprint 1 (October 12 → October 25) · Lab 03 opens today

1-hour lecture · 4 lessons · 4 activities · then the Team Studio (120 min) with Lab 03

Educational material · © 2026 Cyber.SoHo · licensed to you for your personal study and the activities of this course.
Legal Notice, Terms of Use and Acceptable Use Policy — *Student deck · one concept per slide · the notes carry short study notes and the flash-card answers.*

Today in one hour

Time	Part
0:00–0:02	Check-in
0:02–0:12	Lesson 1 · Stories, not specifications
0:12–0:17	Activity 1 · rewrite two requirements
0:17–0:27	Lesson 2 · INVEST and splitting
0:27–0:32	Activity 2 · fun-quiz ×3: which letter fails?
0:32–0:42	Lesson 3 · Acceptance criteria and story points
0:42–0:47	Activity 3 · Planning Poker with fingers
0:47–0:57	Lesson 4 · Velocity, capacity, burndown, forecast
0:57–1:00	Activity 4 · the ideal burndown line + zen-break

Two reminders

Lab 03 — Sprint 1 Kit & User Story Set (D4 + D5) opens today at the start of the Team Studio; feedback date Sunday, October 25, 23:59.
CQ2 (Checkpoint Quiz 2) next Monday, October 26: one velocity computation and one burndown reading — both in Lesson 4.

Team Menu

The fictional team of Classes 3 and 4 — Amira, Noah (Product Owner today), Sofia (Scrum Master), Karim — provides every example.
Their eight backlog items become stories, pass INVEST, get tests and points, and produce the first velocity, burndown and forecast of the course.

Cyber.SoHo — Educational Hub © 2026 | Schiller International University · Legal Notice · Terms of Use · Acceptable Use Policy

What you will be able to do

- Write a Product Backlog item as a user story — As a (role), I want (goal), so that (benefit) — and explain the three Cs: Card, Conversation, Confirmation.
- Test a story with the six **INVEST** (Independent, Negotiable, Valuable, Estimable, Small, Testable) questions and split a story that fails them.
- Write a one-line acceptance test — *Given · When · Then* — with an observable result, and tell it from the **DoD** (Definition of Done).
- Estimate stories in relative story points on a Fibonacci-like scale with Planning Poker, and say why points are not hours.
- Compute a velocity, a capacity in hours and a release forecast; draw and read a burndown chart; rank stories with **MoSCoW** (Must, Should, Could, Won't have this time).

A note on sources: the Scrum Guide 2020 requires a Product Backlog and its refinement — description, order, size. Stories, points, velocity and burndowns are common practices around Scrum, each cited with its author.

Cyber.SoHo — Educational Hub © 2026 | Schiller International University · Legal Notice · Terms of Use · Acceptable Use Policy

Lesson 1 · Stories, not specifications

BY THE END OF THIS LESSON YOU CAN

- State the three parts of a user story and the rule that governs each part: a named user, an action without technique, a benefit that differs from the action.
- Explain the three Cs — Card, Conversation, Confirmation — and why the conversation carries more understanding than the card.
- Tell an epic from a story and a story from a task, and say where each one lives in Scrum's artifacts.

Cyber.SoHo — Educational Hub © 2026 | Schiller International University · Legal Notice · Terms of Use · Acceptable Use Policy

From a backlog item to a user story

The Scrum Guide 2020 on refinement

"Product Backlog refinement is the act of breaking down and further defining Product Backlog items into smaller more precise items … to add details, such as a **description**, **order**, and **size**."
— Schwaber & Sutherland, scrumguides.org · CC BY-SA 4.0

What refinement must produce — and what this hour does

Description → Lessons 1 to 3 (story, INVEST, acceptance test)
Order → Lesson 4 (MoSCoW)
Size → Lessons 3 and 4 (points, velocity)
The Guide says *what*; the user story is the most used *how*.

As a [role], **I want** [goal], **so that** [benefit].

Mike Cohn, *User Stories Applied* (2004), made the form standard; the practice comes from Extreme Programming (late 1990s). Mountain Goat Software — User Stories · Agile Alliance glossary

Cyber.SoHo — Educational Hub © 2026 | Schiller International University · Legal Notice · Terms of Use · Acceptable Use Policy

Three parts, three rules

Part	What it is	The rule	Fails the rule
As a [role]	One real user of the product — one of the users of your D2 (Product Vision Board)	Never “the system”, “the team”, “the admin”, “everyone”	“As the system, I want …”
I want [goal]	An action or an ability the user wants	Names what the user does, not the technique that will deliver it	“a dropdown with six colours”
so that [benefit]	Why the user wants the goal	Must not repeat the goal — it is what lets the team find a cheaper way	“a search box so that I can search”

Why one sentence?

A story is deliberately short: a placeholder for *who*, *what* and *why* that fits an index card and leaves the details to a conversation at the latest responsible moment. It is not a specification — Class 2 showed what writing everything down in advance produced.

Cyber.SoHo — Educational Hub © 2026 | Schiller International University · Legal Notice · Terms of Use · Acceptable Use Policy

The three Cs: Card, Conversation, Confirmation

Card

The written sentence — a reminder of a promise to talk, small enough to be moved around a board.
The smallest of the three.

Conversation

Product Owner and Developers discover the details: what does “today” mean before the kitchen enters the meals? weekends? always three meals?
During refinement, at Sprint Planning, and while the story is built.

Confirmation

The acceptance test: the agreed way to show that the story was implemented correctly.
In this course: one line, Given · When · Then (Lesson 3).

Source: Ron Jeffries, Essential XP: Card, Conversation, Confirmation (2001). Jeffries is one of the three founders of XP (Extreme Programming).

Cyber.SoHo — Educational Hub © 2026 | Schiller International University · Legal Notice · Terms of Use · Acceptable Use Policy

Why the conversation carries more than the card

- The **order matters**: the card is the smallest part; most of the understanding is created in the conversation; the confirmation records what was agreed so that it is not lost.
- A document transfers **less understanding** than a conversation between the person who needs the feature and the people who build it — and a document cannot answer a question.
- That is why a story is **not a specification**: the card is the start of the conversation, not its replacement (Mike Cohn's video).
- For your team: a card with **no conversation behind it is an empty promise**. Lab 03 asks you to read every acceptance test aloud to a member who did not write it — the shortest conversation that exists.

Team Menu · story 1, three answers that are nowhere on the card

Before 10:30 the page shows yesterday's meals with a "not yet updated" line.
On Saturday and Sunday it shows "closed" and Monday's meals if they are known.
Three meals on a normal day — but the page must not break with two or four.

Ten minutes of conversation, Noah with the three Developers.

Cyber.SoHo — Educational Hub © 2026 | Schiller International University · Legal Notice · Terms of Use · Acceptable Use Policy

Three sizes: epic → stories → tasks

Size	What it is	Where it lives	Team Menu example
Epic	A story too big for one Sprint; several benefits hidden inside — split before it is estimated	Product Backlog, until it is split	"The kitchen keeps the page right itself" (item 8: sign-in + correction + log)
Story	One benefit for one user, finished in well under one Sprint	Product Backlog → Sprint Backlog when selected	"As the head chef, I want to correct one allergen entry myself, so that the page is right without waiting for students"
Task	A work item of two hours or less (in this course) that builds part of a story and delivers no benefit on its own	Sprint Backlog — the plan, the "how" of Class 4	"Second-person check against the recipe cards (1 h)"

The sentence to keep

Only stories are estimated, and only Done stories count in the velocity (Lesson 4): an epic is too uncertain to size, and a task is finished without delivering anything a user can use.

Cyber.SoHo — Educational Hub © 2026 | Schiller International University · Legal Notice · Terms of Use · Acceptable Use Policy

Activity 1 · Rewrite two requirements

Requirement 1 — the old way

“The system shall have a login”

Requirement 2 — the old way

“Add a search bar”

- Turn each one into a user story — for **your own product** if it fits, otherwise for Team Menu's allergen page.
- A **real role** (one of your D2 users), a **goal without technique**, a **benefit that does not repeat the goal**.
- Test: can a second person say who the user is, what they can do afterwards, and what problem it solves — without the word “system” and without naming a screen element?
- Two minutes to write, two to swap with the pair next to you, one minute of answers.

Three failures to catch

“As a user …” — nobody in particular
“… so that I can log in” — the benefit repeats the goal
“I want a button that …” — a technique
Your rewrites are reused in Lab 03 (Box 1).

Cyber.SoHo — Educational Hub © 2026 | Schiller International University · Legal Notice · Terms of Use · Acceptable Use Policy

Team Menu · from plain items to stories (1/2)

#	Class 4 item (plain language)	Story (Lab 03, Box 1)
1	3 hot meals + allergens	As a student with an allergy, I want to see today's three hot meals on my phone, so that I can choose before I join the line.
2	Show sesame on every dish	As a student allergic to sesame, I want sesame shown on every dish that contains it, so that I do not eat it by mistake.
3	Icons readable on a phone	As a student in a hurry, I want an icon next to each allergen, so that I understand the page at a glance.
4	French version	As a French-speaking student, I want the page in French, so that I understand the allergens in my own language.
5	Filter: no peanuts / tree nuts	As a student with an allergy, I want to hide dishes that contain my allergen, so that I only see dishes that are safe for me.
6	Weekly menu view	As a student who plans ahead, I want to see the meals of Monday to Friday, so that I can plan my week.
7	Daily-specials banner	As the cafeteria manager, I want a banner for changes of the day, so that students know when a dish is replaced.
8	Kitchen edits an entry in under 2 minutes	As the head chef, I want to correct an allergen entry myself, so that the page is right without waiting for students.

The users come from D2 (Product Vision Board): no story may start with any other "as a".

Cyber.SoHo — Educational Hub © 2026 | Schiller International University · Legal Notice · Terms of Use · Acceptable Use Policy

Team Menu · the three Cs on story 1 (2/2)

Card

As a student with an allergy, I want to see today's three hot meals on my phone, so that I can choose before I join the line.

Conversation (10 min)

Before 10:30: yesterday's meals + “not yet updated”.
Weekend: “closed”, and Monday's meals if known.
Three meals normally — two or four must not break the page.

Confirmation

Given a weekday with three hot meals entered, **when** I open the page on a phone, **then** the three meal names are shown without scrolling sideways.
Observable: count three names; nothing scrolls sideways.

A bad draft, caught by the rules (Karim, item 8)

“As the system, I want an admin page with a dropdown of allergens, so that there is an admin page.” → rule 1 fails (“the system”), rule 2 fails (“dropdown”), rule 3 fails (the benefit repeats the goal). Rewritten in two minutes once the rules were on the board.

Cyber.SoHo — Educational Hub © 2026 | Schiller International University · Legal Notice · Terms of Use · Acceptable Use Policy

Lesson 1 · six flash-cards

1

The three parts of a user story, and the rule of each?

2

The three Cs of a story (Ron Jeffries, 2001)?

3

Why is the card deliberately short?

4

Does the Scrum Guide require user stories?

5

What is an epic, and what happens to it?

6

Story versus task?

Lesson 2 · INVEST and splitting

BY THE END OF THIS LESSON YOU CAN

- Ask the six INVEST questions of any story and name the fix for each "no".
- Apply the lab rule — at most one ✕ per story, with a written reason — and write a spike when a story cannot be estimated.
- Split a story that is too big with the usual patterns, keeping every slice valuable.

Cyber.SoHo — Educational Hub © 2026 | Schiller International University · Legal Notice · Terms of Use · Acceptable Use Policy

INVEST: six questions for every story (1/2)

Letter	The question	The fix when the answer is “no”
Independent	Can it be built and delivered without waiting for another story?	Re-order the backlog, or merge two stories that cannot be separated.
Negotiable	Does it leave the *how* open for the conversation?	Delete the technical details; keep the benefit.
Valuable	Does a user or a customer gain something from it alone?	Rewrite the *so that*; if you cannot, it is a task — attach it to a story.

Read each letter as a question, and each “no” as a signal that names its own fix. Source: Bill Wake, INVEST in Good Stories, and SMART Tasks (2003).

Cyber.SoHo — Educational Hub © 2026 | Schiller International University · Legal Notice · Terms of Use · Acceptable Use Policy

INVEST: six questions for every story (2/2)

Letter	The question	The fix when the answer is “no”
Estimable	Can the Developers say roughly how big it is?	Run a time-boxed research task — a **spike** — then estimate.
Small	Can it be finished in well under one Sprint?	Split it (2.3).
Testable	Can another person decide by looking whether it is done?	Write the acceptance test (Lesson 3); if you cannot, the story is too vague.

Two letters beginners confuse

Independent is about delivery, not code: two stories may share a table and still be shown to a user separately. **Valuable** is judged from the user's side: “As a developer, I want to refactor the database” names no benefit for anybody outside the team — write the benefit users get (pages that load in under two seconds) and keep the refactoring as the task that delivers it.

Cyber.SoHo — Educational Hub © 2026 | Schiller International University · Legal Notice · Terms of Use · Acceptable Use Policy

At most one ✕ — and the spike

The Lab 03 rule

Not six ✓ on every story: **at most one ✕, with a written reason**.
A backlog where every story passes every check was probably not checked.
One ✕ = a known risk, written down — "E ✕: we have never built a phone filter; spike of one hour" — and a known risk is manageable.
Two ✕ on the same story: rewrite it before it is estimated.

The spike — the fix for E

A short, **time-boxed** piece of research whose only output is **knowledge**: one hour to try the phone filter on twelve test dishes, then a report on Wednesday.
Not a story — it delivers nothing to a user — and **not counted in the velocity**.
Written with a time-box and an owner; run before the story is sized; then estimate with what you learned.

Cyber.SoHo — Educational Hub © 2026 | Schiller International University · Legal Notice · Terms of Use · Acceptable Use Policy

Splitting a story that is too big

Pattern	Team Menu example
By workflow step	The chef *signs in* · the chef *corrects one entry* — two stories
By business rule	*Every correction is logged with date and time* — a story of its own
By data variation	Monday's meals before the whole week; English before French
Happy path vs. edge cases	Three meals first; two or four meals, a closed day later
By user or role	The sesame-allergic student before every allergy

The rule older than the patterns

Every slice must still be a **story**: one benefit for one user, demonstrable on its own.
“The database part” is a **technical layer** — layers deliver nothing until all exist. A good slice is a **vertical slice**: thin, but complete from the user's action to the result.

Mike Cohn's SPIDR (Spikes, Paths, Interfaces, Data, Rules) names the same ideas: Story Splitting. After a split, Lab 03 keeps eight stories; the new slices go to the later list.

Cyber.SoHo — Educational Hub © 2026 | Schiller International University · Legal Notice · Terms of Use · Acceptable Use Policy

Activity 2 · Fun-quiz ×3 — which letter fails first?

Story 1

“As a user I want the whole admin module.”

Story 2

“As a user I want the app to be nice.”

Story 3

“As a developer I want to refactor the database.”

The six letters

I Independent
N Negotiable
V Valuable
E Estimable
S Small
T Testable

For each story: which letter fails first — the letter whose fix you would apply before any other — and what is the fix? Then the professor gives the answer in one sentence. Last minute: run the six questions on one story of your own product, in your head.

Cyber.SoHo — Educational Hub © 2026 | Schiller International University · Legal Notice · Terms of Use · Acceptable Use Policy

Team Menu · the INVEST matrix

#	I want …	I	N	V	E	S	T	Verdict	Reason for the kept ×
1	see today's three hot meals on my phone	✓	✓	✓	✓	✓	✓	all ✓	
2	sesame shown on every dish that contains it	✓	✓	✓	✓	✓	✓	all ✓	
3	an icon next to each allergen	✓	✓	✓	✓	✓	✓	all ✓	
4	the page in French	✓	✓	✓	✓	✓	✓	all ✓	
5	hide dishes that contain my allergen	✓	✓	✓	✖	✓	✓	E ×	never built a phone filter → spike 1 h
6	see the meals of Monday to Friday	✓	✓	✓	✓	✓	✓	all ✓	
7	a banner for changes of the day	✓	✓	✓	✓	✓	✓	all ✓	
8	correct an allergen entry myself	✓	✓	✓	✓	✖	✓	S ×	needs a sign-in → split, later list

Two × in eight stories, each with a written reason: a checked backlog, with two managed risks.

Cyber.SoHo — Educational Hub © 2026 | Schiller International University · Legal Notice · Terms of Use · Acceptable Use Policy

Team Menu · split story 8, spike story 5

Diagram 2 · INVEST (Bill Wake, 2003) and how to split a story

Six questions for every story · a fix when the answer is 'no'

I — Independent
Can it be built and delivered without waiting for another
fix: Re-order, or merge two stories that cannot be separated.

N — Negotiable
Does it leave the how open for the Conversation?
fix: Delete the technical details; keep the benefit.

V — Valuable
Does a user or a customer gain something from it alone?
fix: Rewrite the so-that; if impossible, it is a task — attach it to a story.

E — Estimable
Can the Developers say roughly how big it is?
fix: Run a time-boxed research task (a spike), then estimate.

S — Small
Can it be finished in well under one Sprint?
fix: Split it: by step, by rule, by data, by path, by user.

T — Testable
Can another person decide by looking whether it is done?
fix: Write the acceptance test; if you cannot, the story is too vague.

Team Menu's eight stories (Lab 03)

	I	N	V	E	S	T	verdict
#1 See today's three hot meals	✓	✓	✓	✓	✓	✓	all ✓
#2 Sesame shown on every dish	✓	✓	✓	✓	✓	✓	all ✓
#3 An icon next to each allergen	✓	✓	✓	✓	✓	✓	all ✓
#4 The page in French	✓	✓	✓	✓	✓	✓	all ✓
#5 Hide dishes with my allergen	✓	✓	✓	×	✓	✓	E ×
#6 Meals of Monday to Friday	✓	✓	✓	✓	✓	✓	all ✓
#7 Banner for changes of the da	✓	✓	✓	✓	✓	✓	all ✓
#8 Chef corrects an allergen ent	✓	✓	✓	✓	×	✓	S ×

Rule of the lab: at most one × per story, with a written reason

Six splitting patterns · keep every slice valuable (a vertical slice, never a technical layer)

by workflow step
Which steps does the user take, one after the other?
Team Menu: Chef signs in → chef corrects one entry

by business rule
Which rules apply, one at a time?
Team Menu: Every correction is logged with date and time

by data variation
Which kinds of data can be delivered first?
Team Menu: Monday's meals before the whole week

happy path vs. edge cases
The normal case first; the exceptions later.
Team Menu: Three meals first; two or four meals later

by user or role
Which user can be served first?
Team Menu: The sesame-allergic student before every allergy

spike first, then story
What must be learned before sizing?
Team Menu: 1-hour spike: a phone filter on 12 test dishes

Highlighted: the patterns Team Menu used on story 8 (split) and story 5 (spike). Mike Cohn's SPIDR list names the same ideas: Spikes, Paths, Interfaces, Data, Rules.

Cyber.SoHo — Educational Hub © 2026 | Schiller International University · Legal Notice · Terms of Use · Acceptable Use Policy

Lesson 2 · six flash-cards

1

The six INVEST questions?

2

Who published INVEST, and when?

3

The lab rule for INVEST?

4

What is a spike?

5

What makes a slice of a split story a good slice?

6

Five splitting patterns?

Cyber.SoHo — Educational Hub © 2026 | Schiller International University · Legal Notice · Terms of Use · Acceptable Use Policy

Lesson 3 · Acceptance criteria and story points

BY THE END OF THIS LESSON YOU CAN

- Write a one-line acceptance test in the form Given · When · Then with an observable result, and distinguish it from the Definition of Done.
- Explain what a story point measures, why relative sizes are more stable than hours, and why the scale 1, 2, 3, 5, 8, 13 has growing gaps.
- Run Planning Poker with the course protocol — a reference story, a simultaneous reveal, explanations from the extremes, at most two re-votes.

Cyber.SoHo — Educational Hub © 2026 | Schiller International University · Legal Notice · Terms of Use · Acceptable Use Policy

Given · When · Then

Given [the starting situation], **when** [one action], **then** [one observable result].

Observable — the quality of the whole line sits in the last word

A person or a tool can **see it, count it or measure it** without asking the author.
✓ three meal names are shown · the word “sesame” appears · no dish with milk among the twelve test dishes · 3 of 3 test users name the icons
× “the page works well” · “the filter is fast enough” — nobody can film “well”; “fast enough” has no number.
A number is the surest way; when no number fits, a visible fact is enough.

Where the words come from

Dan North, **Introducing BDD (2006)** — Behaviour-Driven Development. The same three words are the keywords of Gherkin, the language of test tools such as Cucumber (reference). You borrow the words, not the tools.
Team Menu, story 2: Given a dish whose recipe card lists sesame, when I open that dish, then the word “sesame” appears in its allergen list.

Cyber.SoHo — Educational Hub © 2026 | Schiller International University · Legal Notice · Terms of Use · Acceptable Use Policy

Acceptance test versus Definition of Done

Acceptance test

Written **per story**.
Answers: *does this story do what it is meant to do?*
One line: Given · When · Then, with an observable result.
Team Menu's story 2: sesame appears on every dish that contains it.

Definition of Done (Class 4)

The **same five checks for every story**.
Answers: *is it finished to our quality standard?*
Second-person check · two phones · spell-check · shared folder · card moved.
Lab 03: exactly five yes-or-no checks and the rule sentence.

The two can disagree — which is why a story needs both

Story 2 can **pass its test and not be Done** (nobody compared the entries with the recipe cards) — and it can **pass all five checks and fail its test** (every entry checked, but the sesame label missing on one dish).

Cyber.SoHo — Educational Hub © 2026 | Schiller International University · Legal Notice · Terms of Use · Acceptable Use Policy

Relative sizing: points, not hours

- A **story point** is a unit of *relative size*: effort + complexity + uncertainty — deliberately **not** a number of hours.
- **Hours depend on who** does the work: the same story takes Karim four hours and Amira seven — an estimate in hours is an estimate of a person.
- People are **poor at absolute estimates and good at relative ones**: “how many hours?” spreads widely; “about twice the sesame story?” agrees quickly.
- A number of hours becomes **a promise** the moment a manager reads it — and a promise distorts the next estimate.
- Points are **relative to a reference**: the smallest well-understood story is 1; everything else is sized against it.
- A velocity in points belongs to **one team**: never compared between teams, never a measure of performance.

Team Menu's reference

Story 2 = 1 point (sesame shown on every dish).
“How many sesames is the French version?” — the answer comes as a comparison: 3.

T-shirt sizes

XS · S · M · L · XL — a coarser scale for early sizing; the same idea with fewer steps.
In this course: points on the scale of the next slide.

Cyber.SoHo — Educational Hub © 2026 | Schiller International University · Legal Notice · Terms of Use · Acceptable Use Policy

The scale: 1 · 2 · 3 · 5 · 8 · 13

1	2	3	5	8	13
XS	S	M	L	XL	XXL
the reference	about twice	about three times	clearly bigger	the largest for a Sprint	too big — split

- The **uncertainty grows with the size**: a team can tell a 2 from a 3, but not a 50 from a 53 — a story of that size hides unknowns worth more than three points.
- Growing gaps stop the team **pretending to a precision it does not have**: a story that feels like “11” is either an 8 with a spike or a 13 to be split.
- **In this course the scale stops at 8**: anything bigger fails S (Small) and is split.

Cyber.SoHo — Educational Hub © 2026 | Schiller International University · Legal Notice · Terms of Use · Acceptable Use Policy

Planning Poker: the protocol

Step	What happens
1	The Product Owner reads the story **and its test**.
2	The Developers ask questions — about thirty seconds.
3	At a signal, **everybody shows a number at the same moment** — nobody speaks a number first.
4	Within one step of each other on the scale → take the **majority** value.
5	More than one step apart (a 3 and an 8) → the **lowest and the highest explain**, 30 s each; vote again, at most twice; still apart → take the higher.

Sources: James Grenning, Planning Poker (2002) · Mountain Goat Software, Planning Poker. The Product Owner's knowledge of value belongs in the explanation, not in the number — a story is not bigger because it is more wanted.

Anchoring

Once the first person says “five”, every later estimate moves towards five — a documented effect in the study of judgment. The simultaneous reveal exists to defeat it.

With fingers (class and Lab 03)

1, 2, 3 fingers = 1, 2, 3 points
open hand = 5 · closed fist = 8
flat hand turned sideways = “I cannot estimate this yet” → spike

The conversation is the product of the game: a spread between 3 and 8 means one person knows a risk.

Cyber.SoHo — Educational Hub © 2026 | Schiller International University · Legal Notice · Terms of Use · Acceptable Use Policy

Activity 3 · Planning Poker with fingers

Story 2 — the reference = 1

As a student allergic to sesame, I want sesame shown on every dish that contains it, so that I do not eat it by mistake.
Test: Given a dish whose recipe card lists sesame, when I open that dish, then the word "sesame" appears in its allergen list.

Story 3

As a student in a hurry, I want an icon next to each allergen, so that I understand the page at a glance.
Test: Given a dish with two allergens, when I open it, then two icons appear, and 3 of 3 test users name them correctly.

Story 5

As a student with an allergy, I want to hide dishes that contain my allergen, so that I only see dishes that are safe for me.
Test: Given I chose "milk", when I apply the filter, then no dish with milk is shown among the 12 test dishes.

Protocol

Story + test read aloud · 30 s of questions
"one, two, three" → fingers at the same moment
lowest and highest explain (30 s each) → one more vote
numbers on the board

1, 2, 3 fingers = 1, 2, 3 points · open hand = 5 · closed fist = 8 · flat hand sideways = cannot estimate yet (→ spike). This is exactly Step 3 of Lab 03, played on your own eight stories in the studio.

Cyber.SoHo — Educational Hub © 2026 | Schiller International University · Legal Notice · Terms of Use · Acceptable Use Policy

Team Menu · tests and votes

Diagram 3 · Acceptance test, the point scale, and Planning Poker

Acceptance test · Given · When · Then (Dan North, 2006)

Given the starting situation — a weekday with three meals entered,

When one action — I open the page on a phone,

Then one observable result — the three meal names are shown without scrolling sideways — a number or a visible fact, never 'works well'.

Story points · a Fibonacci-like scale (relative size, not hours)

1	2	3	5	8	13
XS	S	M	L	XL	XXL

Gaps grow because uncertainty grows with size: 5 and 8 differ; 50 and 53 do not.

Planning Poker (James Grenning, 2002) · the course protocol with fingers

1 · Reference — The smallest story is called 1 point. Size everything against it.

→ **2 · Read + ask** — The PO reads the story and its test; the Developers ask questions — 30 s.

→ **3 · Show at once** — 'One, two, three': everybody shows fingers at the same moment — no anchoring.

→ **4 · Spread?** — More than one step apart (3 vs 8)? Lowest and highest explain, 30 s each.

→ **5 · Re-vote** — At most two re-votes; then take the higher number. Record the disagreement word.

Fingers: 1, 2, 3 = 1, 2, 3 points · open hand = 5 · fist = 8 · flat hand sideways = 'I cannot estimate yet' → the story fails E and gets a spike

Team Menu's eight rounds (Lab 03, Step 3)

story	round 1	round 2	points	what happened
#1 See today's three hot meals	3 3 2 3	—	3	adjacent votes → majority
#2 Sesame shown on every dish	1 1 1 1	—	1	reference story = 1 point
#3 An icon next to each allergen	2 2 3 2	—	2	adjacent votes → majority
#4 The page in French	3 5 3 3	—	3	adjacent votes → majority
#5 Hide dishes with my allergen	3 5 8 5	5 5 8 5	5	spread 2 steps → extremes explain → re-vote; spike kept
#6 Meals of Monday to Friday	3 3 3 5	—	3	adjacent votes → majority
#7 Banner for changes of the day	2 1 2 2	—	2	adjacent votes → majority
#8 Chef corrects an allergen entry	5 8 8 5	—	5	adjacent votes → the team settled on 5; S × kept

Total = 3 + 1 + 2 + 3 + 5 + 3 + 2 + 5 = 24 points. Points are relative to the 1-point reference; a velocity in points is never compared between teams.

Cyber.SoHo — Educational Hub © 2026 | Schiller International University · Legal Notice · Terms of Use · Acceptable Use Policy

Lesson 3 · six flash-cards

1

The form of an acceptance test in this course?

2

What makes a "then" observable?

3

Acceptance test versus Definition of Done?

4

What does a story point measure, and what is it not?

5

Why do the gaps of the scale 1, 2, 3, 5, 8, 13 grow?

6

What does the simultaneous reveal of Planning Poker prevent?

Cyber.SoHo — Educational Hub © 2026 | Schiller International University · Legal Notice · Terms of Use · Acceptable Use Policy

Lesson 4 · Velocity, capacity, burndown and the release forecast

BY THE END OF THIS LESSON YOU CAN

- Compute a velocity from past Sprints and a release forecast in Sprints, rounded up; compute a capacity in hours and say why it is a different number.
- Draw the ideal line of a burndown chart from the Sprint total and read the actual line: ahead, behind, flat then drop.
- Rank stories with MoSCoW, keep the Must haves to about 60 % of the points, and compute the planned, cautious and updated forecasts of Lab 03.

Cyber.SoHo — Educational Hub © 2026 | Schiller International University · Legal Notice · Terms of Use · Acceptable Use Policy

Velocity

Definition

Velocity = the sum of the story points of the stories that **met the DoD** in a Sprint.
A story partly finished counts **zero**, whatever its progress.

Class example

18 + 22 + 20 = 60 ÷ 3 = **20 points per Sprint**
120 points left → 120 ÷ 20 = **6 Sprints**
130 points → 6.5 → **7 Sprints** (round up)

- For forecasting, use the **average of the last three Sprints**; when the division is not whole, **round up** — a partial Sprint costs a whole Sprint.
- **One team, its own scale**: never compared between teams. **Not a target**: a team asked to raise its velocity raises its estimates. **A forecast, not a promise**: the Guide says the Developers *forecast*.
- A first Sprint has no history: Lab 03 uses your **planned velocity** (the points planned into Sprint 1) and replaces it with the real one when the Sprint ends.

Cyber.SoHo — Educational Hub © 2026 | Schiller International University · Legal Notice · Terms of Use · Acceptable Use Policy

Capacity in hours

Capacity

How much work the team **can** do in a Sprint, in **hours**: **people × focused hours per day × working days**

Two examples

Class example: 4 developers × 6 focused hours (not 8: meetings, mail, interruptions) × 9 working days (ten minus one for the events) = **216 h**
Team Menu (Class 4): 4 students × 4 h a week × 2 weeks − 3 h of events = **29 h**

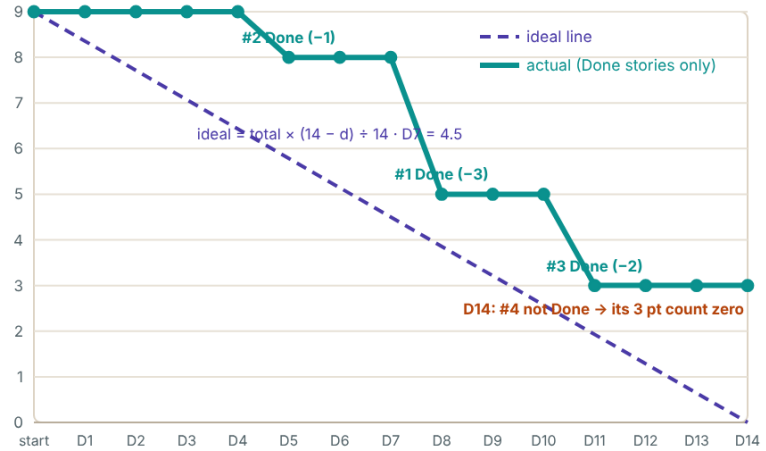
Capacity and velocity are used side by side — never converted

Capacity tells the Developers how much *time* they have at Sprint Planning (does the plan fit the calendar?). **Velocity** tells them how many *points* they finished before (does it fit the team's history?). Converting one into the other would turn points back into hours.

Cyber.SoHo — Educational Hub © 2026 | Schiller International University · Legal Notice · Terms of Use · Acceptable Use Policy

The burndown chart

Diagram 4 · Sprint 1 burndown, MoSCoW and the release forecast (Team Menu)



Reading: the actual line stayed above the ideal line all Sprint and dropped in three steps — stories were finished one at a time at the end of each week, and too many were started at once. A scope change gets a written note.

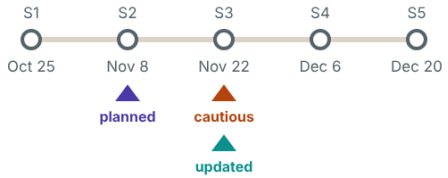
MoSCoW · 24 pt · Must 7 (29 %) · Should 9 · Could 3 · Won't 5 · Must ≤ 60 % ✓



Release scope = Must + Should = 16 pt.
Sprint 1 = #1, #2, #4, #3 = 9 pt = burndown total = planned velocity.

Velocity = points of the stories that met all five checks of the Definition of Done in the Sprint; partly finished stories count zero.
Forecast: remaining ÷ velocity, rounded UP; cautious velocity = planned × 0.7, rounded DOWN to a whole number. Never compare velocities between teams.

Release forecast · remaining ÷ velocity, up



Planned
remaining = 16 − 9 = 7
7 ÷ 9 = 0.78 → 1 Sprint
→ ends Sprint 2, Nov 8

Cautious (velocity × 0.7)
9 × 0.7 = 6.3 → 6 (rounded down)
7 ÷ 6 = 1.17 → 2 Sprints
→ ends Sprint 3, Nov 22

Updated (real velocity)
Done: #2, #1, #3 = 6 pt (#4 = 0)
remaining = 16 − 6 = 10
10 ÷ 6 = 1.67 → 2 Sprints → Nov 22

A forecast is not a promise: it is re-computed every Sprint.

The class example
velocity = (18 + 22 + 20) ÷ 3 = 20 pt per Sprint
120 pt left ÷ 20 = 6 Sprints
capacity = 4 devs × 6 h × 9 days = 216 h
capacity = hours available; velocity = points Done

The ideal line

A straight line from the Sprint total on day 0 to zero on the last day.
ideal(d) = **total × (days − d) ÷ days**
40 points, 10 days → loses 4 a day; Team Menu: 9 × (14 − d) ÷ 14.

The actual line

Drawn **every day from the board**: total − the points of the stories that met the DoD.
A partly finished story has **not reduced the line at all**.

Reading a burndown

What you see	What it means	What to do
Actual line **above** the ideal line	The team is **behind**	The Daily Scrum turns “behind” into a changed plan
Actual line **below** the ideal line	The team is **ahead**	Check that nothing was counted before it met the DoD
Flat for days, then a drop	Stories are finished one at a time, at the end — too many started at once	Class 6: the WIP (Work In Progress) limit
The line **goes up**	Scope was added during the Sprint	Allowed only with the Product Owner, never against the Sprint Goal; write a note on the day

Team Menu's reading, in two sentences

The actual line stayed above the ideal line for the whole Sprint and dropped in three steps, because we finish stories one at a time at the end of a week; we started too many stories at once. The Guide: burndowns “do not replace the importance of empiricism”.

Cyber.SoHo — Educational Hub © 2026 | Schiller International University · Legal Notice · Terms of Use · Acceptable Use Policy

MoSCoW

Letter	Meaning	The test question	Team Menu
Must have	The release is not worth delivering without it	If we do not get this, do we cancel the release?	stories 1, 2, 4 = 7 points (29 %)
Should have	Important, painful to leave out, a workaround exists	Could users manage one more Sprint without it?	stories 3, 5, 7 = 9 points
Could have	Desirable; the first to drop when time is short	Would anyone complain loudly if it were missing?	story 6 = 3 points
Won't have this time	Deliberately not built now — written with a reason	Which story do we deliberately not build now?	story 8 = 5 points (needs a sign-in)

Keep the Must haves to a limited share of the effort — around **60 %** as the usual rule of thumb — so that the plan has room when something goes wrong; a release where everything is a Must has no plan, only a list. Source: Agile Business Consortium, MoSCoW Prioritisation.

Cyber.SoHo — Educational Hub © 2026 | Schiller International University · Legal Notice · Terms of Use · Acceptable Use Policy

The release forecast of Lab 03

Line	Formula	Team Menu
Release scope	points of all Must and Should stories	7 + 9 = **16**
Remaining	release scope − Must/Should points in Sprint 1	16 − 9 = **7**
Planned forecast	remaining ÷ planned velocity, **rounded up** to whole Sprints	7 ÷ 9 = 0.78 → **1 Sprint** → Sunday, November 8
Cautious forecast	remaining ÷ (0.7 × planned velocity, rounded down), rounded up	9 × 0.7 = 6.3 → 6; 7 ÷ 6 = 1.17 → **2 Sprints** → November 22
Dates	two-week Sprints ending on Sundays	Oct 25 · Nov 8 · Nov 22 · Dec 6 · Dec 20

The sentence that matters

Write one sentence about **what would have to be true for the cautious case to happen** — that sentence is the risk you are managing. Team Menu: *the kitchen's recipe photos keep arriving late and the filter spike shows that story 5 is bigger than 5.*

Cyber.SoHo — Educational Hub © 2026 | Schiller International University · Legal Notice · Terms of Use · Acceptable Use Policy

Team Menu · the numbers

Quantity	Team Menu
Total of the eight stories	24 points
Must 7 · Should 9 · Could 3 · Won't 5	Must share 29 % ✓ (≤ 60 %)
Release scope (Must + Should)	16 points
Sprint 1 (stories 1, 2, 4, 3) = planned velocity	9 points
Planned forecast	7 ÷ 9 → 1 Sprint → **Nov 8**
Cautious forecast	7 ÷ 6 → 2 Sprints → **Nov 22**
Real velocity (stories 2, 1, 3 Done; story 4 not)	**6 points**
Updated forecast	(16 − 6) ÷ 6 = 1.67 → 2 Sprints → **Nov 22**

The burndown, day by day

Day 0: 9 · day 4: still 9 (2.6 behind)
Day 5: story 2 Done → 8
Day 8: story 1 Done → 5
Day 11: story 3 Done → 3
Day 14: story 4 **not Done** → 3 remain; its 3 points count zero
Above the ideal line all Sprint; three steps; too many stories started at once.

Cyber.SoHo — Educational Hub © 2026 | Schiller International University · Legal Notice · Terms of Use · Acceptable Use Policy

Three words from the reading

Technical debt

A shortcut taken now that makes later work slower.
Ward Cunningham, 1992 — the comparison with financial debt (c2.com)
Team Menu: dish names typed by hand twice.
Lab 03's Retrospective asks which shortcut your team took.

CI — Continuous Integration

Developers combine their work into a shared repository **at least daily**, and an automated build with tests checks every combination.
Martin Fowler, Continuous Integration

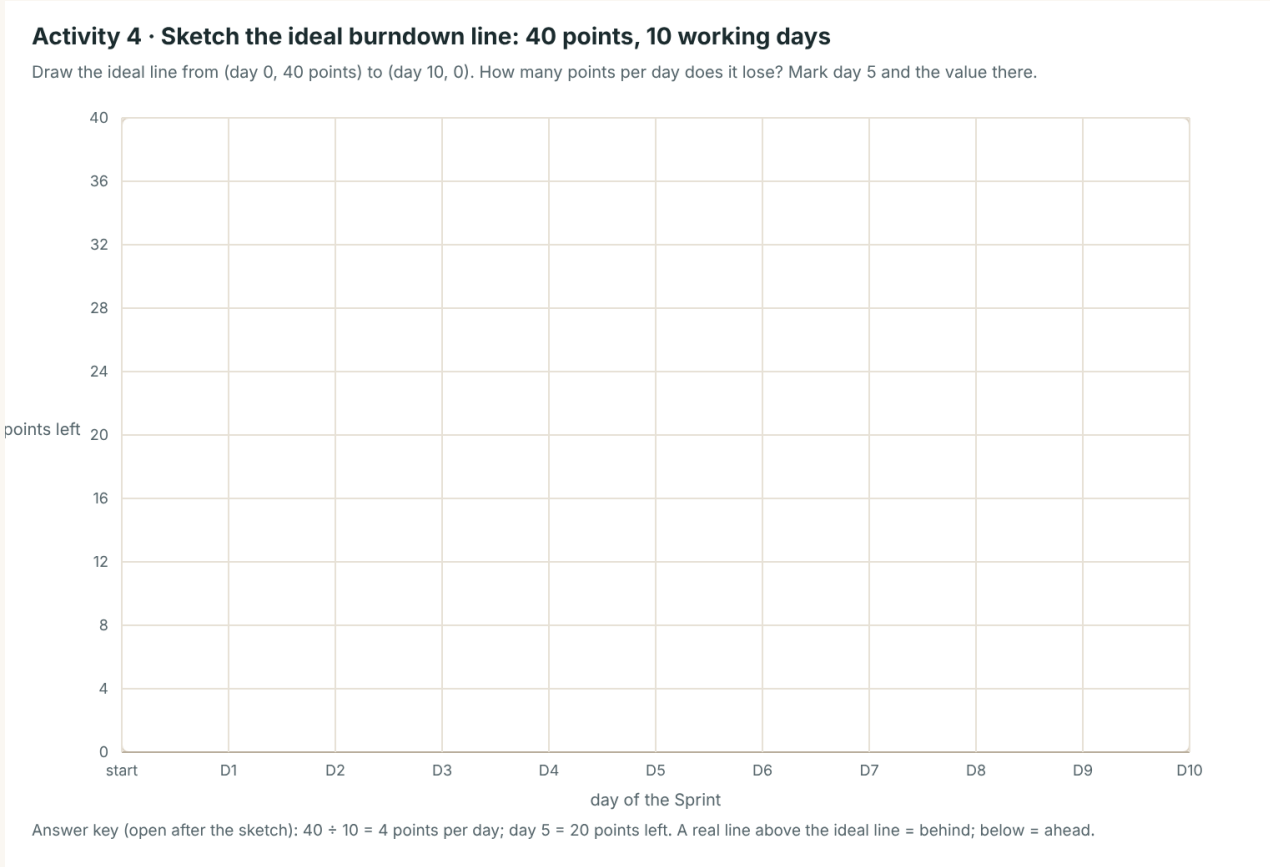
DevOps — Development + Operations

Joins the people who build software and the people who run it, so that software moves from idea to users in **small, frequent steps**.

All three are what makes a two-week Sprint possible at the scale of a company; Class 7 comes back to them as design questions.

Cyber.SoHo — Educational Hub © 2026 | Schiller International University · Legal Notice · Terms of Use · Acceptable Use Policy

Activity 4 · the ideal burndown line + zen-break



- **A 40-point Sprint, ten working days.** Draw the ideal line; write how many points it loses per day; mark the value at day 5.
- Then mark where an actual line would be on day 5 if the team had finished **one 8-point story and started three others**.
- 90 seconds alone · 30 seconds: your neighbour checks the two numbers.
- Then **60 seconds of silence** — no phones, no talking; breathe, stretch. Lab 03 starts in the Team Studio.

Hand-out: 02_schematics/diagram4b_activity4_blank_burndown_grid.svg (or .png) in the companion ZIP. This is the burndown reading of CQ2, with other numbers.

Cyber.SoHo — Educational Hub © 2026 | Schiller International University · Legal Notice · Terms of Use · Acceptable Use Policy

Lesson 4 · six flash-cards

1

What is velocity, and what counts zero?

2

Sprints of 18, 22 and 20 points; 120 points left. Forecast?

3

Capacity: 4 developers × 6 focused hours × 9 days?

4

The ideal line of a burndown, and its value at day d?

5

Reading the actual line?

6

MoSCoW and the 60 % rule?

Cyber.SoHo — Educational Hub © 2026 | Schiller International University · Legal Notice · Terms of Use · Acceptable Use Policy

After the hour · Team Studio = Lab 03

Time-box	What the team does
Planning · 10 min	Today's goal: “Our Product Backlog is ready to be built”; PO = member 2, SM = member 3 of your rotation table
Eight user stories · 40 min	Write alone · INVEST with at most one × per story · one Given · When · Then test each · the Product Owner orders
Definition of Done · 10 min	Exactly five yes-or-no checks and the rule sentence
Planning Poker + MoSCoW · 20 min	Fingers; smallest story = 1; Must points ≤ about 60 % of the total; a reason for each Won't
Sprint Goal and forecast · 15 min	Release scope = Must + Should; remaining ÷ velocity, rounded up; cautious case at 0.7 × velocity; Sprint ends Oct 25 · Nov 8 · Nov 22 · Dec 6 · Dec 20
Review 15 · Retrospective 5 · Plan the week 5	Story 1 in 90 seconds + one INVEST question from the class · start / stop / continue · who writes the Daily Scrum lines, owns the burndown, exports the PDF

Feedback date: Sunday, October 25, 23:59 — the last day of Sprint 1. Final deadline of every lab: Sunday, December 20, 2026, 23:59. Lab 03 goes through SafeAssign (Legal Notice, section 18).

Cyber.SoHo — Educational Hub © 2026 | Schiller International University · Legal Notice · Terms of Use · Acceptable Use Policy

Home Sprint · next Monday

Home Sprint

Three real **Daily Scrum lines** (date · points Done of total · impediment · plan change).
The **Sprint 1 burndown** (day 1 = October 12 … day 14 = October 25; ideal = total × (14 − d) ÷ 14).
Five lines of **Sprint 1 Review and Retrospective** with the real velocity and the updated forecast; export `DIG123_D4D5_<TeamName>_2026-10-25.pdf`.
Read: Denning, *The Age of Agile*, Chapter 5 — technical debt, CI, DevOps (section 4.6).
Your **Learning Log**: five lines.

Next Monday, October 26 — Class 6, Kanban and flow

No new lab.
CQ2 (Checkpoint Quiz 2) in the first 15 minutes of the studio: Scrum events and artifacts, INVEST, story points, **one velocity computation and one burndown reading**.
Your backlog becomes a **Kanban board** with WIP (Work In Progress) limits; the dates on your board become lead-time and cycle-time data — **keep the dates**.
Lab 04 — Flow & Digital Business Brief opens Monday, November 2 (feedback Sunday, November 8). **Mid-Term** November 16: stories with tests, a forecast, a burndown reading — with other numbers.

Cyber.SoHo — Educational Hub © 2026 | Schiller International University · Legal Notice · Terms of Use · Acceptable Use Policy

All links (1/2) · official sites, checked October 16–17, 2026

The Scrum Guide 2020 — scrumguides.org — The official, free definition of Scrum (Schwaber & Sutherland, November 2020). Its sections "Product Backlog" and "Sprint Planning" are the ground of this lecture; it does not prescribe user stories, points, velocity or burndowns.

Creative Commons Attribution-ShareAlike 4.0 — creativecommons.org — The licence of the Scrum Guide: free to share and adapt with attribution, under the same licence.

Scrum.org — Myth 9: Story Points are Required in Scrum — A Scrum.org article explaining that story points, velocity and burndowns are optional practices, not Scrum rules.

Scrum.org — Burndown Charts Defined (video page) — The resource page of the Lesson 4 video, with its published length (4:37); presenter Ralph Jocham, Professional Scrum Trainer.

Ron Jeffries — Essential XP: Card, Conversation, Confirmation (2001) — The article that named the three Cs of a user story.

Agile Alliance — glossary: User Stories — The Agile Alliance definition of a user story, with the 3 Cs and the origin of the practice in Extreme Programming.

Mountain Goat Software — User Stories — Mike Cohn's reference page on the "As a … I want … so that …" form and its use.

Mountain Goat Software — Story Splitting — Mike Cohn's SPIDR approach to splitting stories (Spikes, Paths, Interfaces, Data, Rules).

Mountain Goat Software — Planning Poker — How Planning Poker is played, and why the simultaneous reveal matters.

Mountain Goat Software — videos — The publisher's own video page, where the lengths of the seven Mountain Goat videos of this lecture are published.

Bill Wake — INVEST in Good Stories, and SMART Tasks (2003) — The original INVEST article.

Dan North — Introducing BDD (2006) — Where the words Given, When, Then come from (Behaviour-Driven Development).

Cucumber — Gherkin reference — The official syntax of Gherkin, the language in which Given · When · Then tests are written by tools; you only borrow the three words.

Cyber.SoHo — Educational Hub © 2026 | Schiller International University · Legal Notice · Terms of Use · Acceptable Use Policy

All links (2/2) · official sites, checked October 16–17, 2026

James Grenning — Planning Poker (2002) — The paper that described Planning Poker.

Agile Business Consortium — MoSCoW Prioritisation — The owner of the DSDM framework explains Must, Should, Could, Won't and the limited share of Must haves.

Ward Cunningham — The WyCash Portfolio Management System (1992) — The experience report where the comparison between a technical shortcut and financial debt first appears.

Martin Fowler — Continuous Integration — The reference description of continuous integration.

Atlassian — Agile burndown chart tutorial — A free written explanation of burndown charts, with the usual reading rules.

Stephen Denning — stevedenning.com — Author of The Age of Agile (2018), the required text; Chapter 5 is this week's reading.

VitalSource Bookshelf — vitalsource.com — Where your e-copy of The Age of Agile lives.

Blackboard — blackboard.com — The course platform: attendance, the Lab 03 hand-in, the Q&A board.

tutor.com — Free tutoring and writing review, reached from the Blackboard course (Course and institution tools).

VirusTotal — link check — Paste any link here before you open it (Legal Notice, section 14 and Annex C).

Google Safe Browsing — site status — A second, independent check of a web site's reputation.

Cyber.SoHo — Legal Notice, Terms of Use and Acceptable Use Policy (v2.0) — The rules that govern this material: licence for your personal study, restrictions, no affiliation, external links, recording, academic integrity, personal information.

Videos: What Is a User Story in Agile? (Examples + Template) (about 9 min, L1) · Agile Epic, User Story, and Feature: Do Names Matter? (about 7 min, L1) · How to Split User Stories with SPIDR | 5 Story Splitting Techniques (about 8 min, L2) · The 5 Biggest User Story Mistakes Teams Make (about 7 min, L2) · Definition of Done vs Acceptance Criteria: What's the Difference? (about 5 min, L3) · Planning Poker Explained: How Agile Teams Estimate Story Points (about 6 min, L3) · What is Velocity? Agile Velocity 101 (about 5 min, L4) · Burndown Charts Defined (4:37, L4)

Legal Notice, Terms of Use and Acceptable Use Policy (v2.0): https://cyber-soho.github.io/prevmon/00.01.%20cyber.soho%20-%20legal.html — sections 2 · 4.2–4.6 · 5 · 6.7 · 11 · 14 + Annex C · 17 · 18 · 20 · Annex A (e-signature form, separate file). Scrum Guide quotations © Schwaber & Sutherland, CC BY-SA 4.0. Team Menu is fictional.